ARTIFICIAL INTELLIGENCE

Assessment Tool 1 — Analytical Problem Solving | Answer Guide

1. Water Jug Problem (4-gallon and 3-gallon jugs)

Objective: Obtain exactly 2 gallons in the 4-gallon jug using only the two jugs and a water pump.

Let (x, y) represent the amount of water in the 4-gallon jug and the 3-gallon jug respectively. Initial state: (0, 0).

Step	Action	4-gallon jug	3-gallon jug
1	Fill the 3-gallon jug	0	3
2	Pour into the 4-gallon jug	3	0
3	Fill the 3-gallon jug again	3	3
4	Pour into the 4-gallon jug until full	4	2
5	Empty the 4-gallon jug	0	2
6	Pour remaining water into the 4-gallon jug	2	0

Final state: (2, 0). Exactly 2 gallons are in the 4-gallon jug. The problem is solved by applying a sequence of valid state transitions.

2. Mars Rover as an Intelligent Agent

i. Percepts: Images from cameras; distance and obstacle data; rock and soil composition; temperature, radiation and atmospheric conditions; position, orientation and wheel movement; terrain, slopes and hazards.

ii. Operating environment:

Property	Classification	Reason
Observability	Partially observable	The rover cannot observe every feature or hazard in advance.
Determinism	Stochastic	Unexpected terrain and equipment variations can affect outcomes.
Task structure	Sequential	Current actions influence future navigation and scientific tasks.
Change	Dynamic	Environmental conditions can change while the rover operates.
State variables	Continuous	Position, movement, temperature and time vary continuously.
Agents	Primarily single-agent	It acts independently, though it may coordinate with Earth-based operators.

iii. Actions: Move, turn, stop, avoid obstacles, capture images, record data, inspect or collect samples with robotic instruments, analyze rocks and soil, transmit data to Earth, and manage power.

iv. Performance evaluation: Scientific data quality and quantity; successful sample collection and analysis; area explored; navigation accuracy; obstacle avoidance; efficient power and time use; reliable data transmission; and completion of scientific objectives.

v. Suitable architecture: A model-based, goal-based agent with utility-based decision-making. A model-based component maintains an internal representation of terrain; goal-based planning helps reach targets and collect samples; utility-based decisions balance scientific value, risk, time and energy.

Conclusion: This architecture supports planning and adaptation in a partially observable environment while managing limited resources.

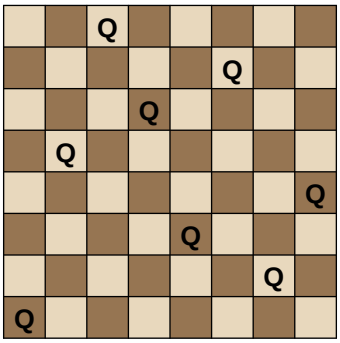
3. Eight-Queens Problem

Objective: Place eight queens on an 8 × 8 chessboard so no two queens share a row, column or diagonal.

Problem formulation: Initial state—empty board. State space—all possible queen arrangements. Actions—place a queen in a safe square. Transition model—add a queen to the chosen square. Goal test—eight queens with no attacks between them. Path cost—one per placement, for a total of eight placements.

Search strategy: Backtracking Search. Place a queen in each column, checking safety. If no safe row exists, remove the previous queen and try another row. Stop when all eight queens are placed.

One valid solution (row numbers for columns 1–8): [1, 5, 8, 6, 3, 7, 2, 4].



Pseudocode:

```
N_QUEENS(board, column):
if column == 8: return SUCCESS
for row = 0 to 7:
if IS_SAFE(board, row, column):
place queen
if N_QUEENS(board, column + 1) == SUCCESS: return SUCCESS
remove queen (backtrack)
return FAILURE
```

Conclusion: Backtracking rejects invalid partial arrangements and systematically searches for a valid solution.

4. OLA Cab Booking Using a Problem-Solving Agent

Suitable agent: Goal-based problem-solving agent. The goal is to reach the destination. Cab selection can consider availability, fare, estimated travel time and customer preference.

Problem formulation: Initial state—customer at pickup location. Goal state—customer reaches destination. Actions—enter locations, view available cabs, select a cab, book and travel. State space—cab types, routes, fares and times. Goal test—destination reached. Path cost—fare, time or a weighted combination.

Pseudocode:

```
OLA_CAB_BOOKING():
INPUT pickup_location, destination
IF pickup_location == destination: PRINT "Already at destination"; RETURN
available_cabs = GET_AVAILABLE_CABS(pickup_location)
IF available_cabs is empty: PRINT "No cabs available"; RETURN
DISPLAY available_cabs
INPUT preferred_cab_type, customer_priority
routes = GET_ROUTES(pickup_location, destination)
FOR each cab in available_cabs:
cab.fare = ESTIMATE_FARE(cab, routes)
cab.time = ESTIMATE_TIME(cab, routes)
selected_cab = SELECT_BEST_OPTION(available_cabs, customer_priority)
IF selected_cab is available:
CONFIRM_BOOKING(selected_cab)
TRACK_RIDE()
IF destination is reached: PRINT "Ride completed successfully"
ELSE: PRINT "Please select another cab"
END
```

Conclusion: The agent helps select a suitable available cab and supports the customer's goal of reaching the destination.

5. Uniform Cost Search (UCS)

Objective: Find the minimum-cost path from S to G. UCS expands the node with the smallest cumulative path cost g(n).

Edge	Cost
S → A	1
S → G	12
A → B	3
A → C	1
B → D	3
C → D	1
C → G	2
D → G	3

Candidate paths:

Path	Calculation	Total cost
S → G	12	12
S → A → B → D → G	1 + 3 + 3 + 3	10
S → A → C → D → G	1 + 1 + 1 + 3	6
S → A → C → G	1 + 1 + 2	4

UCS pseudocode:

```
UNIFORM_COST_SEARCH(start, goal):
priority_queue = [(0, start, [start])]
best_cost = {}
WHILE priority_queue is not empty:
(cost, node, path) = REMOVE_MIN(priority_queue)
IF node == goal: RETURN path, cost
IF node in best_cost AND best_cost[node] <= cost: CONTINUE
best_cost[node] = cost
FOR each (neighbor, edge_cost) of node:
new_cost = cost + edge_cost
IF neighbor not in best_cost OR new_cost < best_cost[neighbor]:
INSERT(priority_queue, (new_cost, neighbor, path + [neighbor]))
RETURN FAILURE
```

Final answer: Optimal path = S → A → C → G. Total cost = 1 + 1 + 2 = **4**.

Conclusion: With non-negative edge costs, UCS finds the least-cost path by expanding the lowest-cost frontier node first.